A

Summer Training Project Report

On

Movie recommendation system(python and ML)

Submitted in partial fulfilment of the requirements

for the award of the degree of

Bachelors of Computer Applications

To

Guru Gobind Singh Indraprastha University, Delhi

Mr. Raman Tiwari

(EXTERAL GUIDE)

MS. . Pooja Rana

(INTERNAL GUIDE)

SUBMITTED BY: Hardik Agrawal

ENROLL NO: 06821102024

COURSE: BCA –E2

SEMESTER:3

Institute of Information Technology & Management

New Delhi – 110058

Acknowledgement

I am profoundly thankful to everyone who contributed to the successful completion of my summer training project.

Firstly, I would like to extend my deepest gratitude to Mr. Raman Tiwari, whose invaluable guidance and insights were essential throughout the training. His expertise and patience were crucial in helping me navigate the complexities of advanced data science and machine learning.

I also wish to sincerely thank IITM Janakpuri for collaborating with Shape My Skills Pvt. Ltd. to provide this exceptional learning opportunity. Special appreciation goes to our course coordinator, Mr. Ashish Nayyar, for his dedicated efforts and support during the training. Furthermore, I am deeply grateful to Mr. Ganesh Wadhwani, Head of the Department, for his encouragement and for providing the necessary resources and environment to facilitate this learning experience.

Finally, I acknowledge the unwavering support of my family and friends, who have been my pillars of strength and encouragement throughout this journey.

Thank you all for making this experience valuable and memorable.

Sincerely,
Hardik Agrawal

Certificate

This is to certify that **Hardik Agrawal** has successfully completed the project titled *"Movie Recommendation system "* as part of the **Machine Learning and Data Science summer training program** organized by **IITM Janakpuri** in collaboration with **ShapeMySkills Pvt. Ltd.**

This project was conducted under the esteemed guidance of **Mr. Raman Tiwari**, whose expertise and mentorship were instrumental in its successful completion. The project exemplifies a thorough understanding of machine learning and data science techniques, highlighting the skills acquired during the training program.

We commend Monisha for her **dedication, hard work, and enthusiasm** throughout the project duration.

Coordinator: Mr. Ashish Nayyar
Head of Department: Mr. Ganesh Wadhwani
Date:
Signature:

INDEX

Chapter-1

Introduction to Python and machine learning

Python is a high-level, interpreted programming language known for its simplicity and versatility. It is widely used in web development, automation, data analysis, artificial intelligence, and scientific computing. One of the major reasons for its popularity in data science and machine learning is its vast ecosystem of libraries and frameworks.

Machine Learning (ML) is a subfield of Artificial Intelligence that focuses on building systems that learn from and make decisions based on data. Rather than being explicitly programmed to perform tasks, ML algorithms learn patterns and relationships from the data they are given.

Types of Machine Learning:

1. Supervised Learning – The algorithm learns from labeled training data and makes predictions.
2. Unsupervised Learning – The algorithm identifies patterns in data without labels.
3. Reinforcement Learning – The algorithm learns through rewards and penalties.

Applications of ML include recommendation systems, image and speech recognition, fraud detection, and autonomous vehicles.

Python language is widely used in Machine Learning because it provides libraries like NumPy, Pandas, Scikit-learn, TensorFlow, and Keras. These libraries offer tools and functions essential for data manipulation, analysis, and building machine learning models. It is well-known for its readability and offers platform independence. These all things make it the perfect language of choice for Machine Learning.

Machine Learning is a subdomain of artificial intelligence. It allows computers to learn and improve from experience without being explicitly programmed, and It is designed in such a way that allows systems to identify patterns, make predictions, and make decisions based on data.

Introduction to Machine Learning

Machine learning (ML) allows computers to learn and make decisions without being explicitly programmed. It involves feeding data into algorithms to identify patterns and make predictions on new data. It is used in various applications like image recognition, speech processing, language translation, recommender systems, etc. In this article, we will see more about ML and its core concepts.

Why do we need Machine Learning?

Traditional programming requires exact instructions and doesn't handle complex tasks like understanding images or language well. It can't efficiently process large amounts of data. Machine Learning solves these problems by learning from examples and making predictions without fixed rules. Let's see various reasons why it is important:

1. Solving Complex Business Problems

Traditional programming struggles with tasks like language understanding and medical diagnosis. ML learns from data and predicts outcomes easily.

Examples:

- Image and speech recognition in healthcare.
- Language translation and sentiment analysis.

2. Handling Large Volumes of Data

The internet generates huge amounts of data every day. Machine Learning processes and analyzes this data quickly by providing valuable insights and real-time predictions.

Examples:

- Fraud detection in financial transactions.
- Personalized feed recommendations on Facebook and Instagram from billions of interactions.

3. Automate Repetitive Tasks

ML automates time-consuming, repetitive tasks with high accuracy hence reducing manual work and errors.

Examples:

- Gmail filtering spam emails automatically.
- Chatbots handling order tracking and password resets.
- Automating large-scale invoice analysis for key insights.

4. Personalized User Experience

ML enhances user experience by tailoring recommendations to individual preferences. It analyze user behavior to deliver highly relevant content.

Examples:

- Netflix suggesting movies and TV shows based on our viewing history.
- E-commerce sites recommending products we're likely to buy.

5. Self Improvement in Performance

ML models evolve and improve with more data helps in making them smarter over time. They adapt to user behavior and increase their performance.

Examples:

- Voice assistants like Siri and Alexa learning our preferences and accents.
- Search engines refining results based on user interaction.

- Self-driving cars improving decisions using millions of miles of driving data.

What Makes a Machine "Learn"?

A machine "learns" by identifying patterns in data and improving its ability to perform specific tasks without being explicitly programmed for every scenario. This learning process helps machines to make accurate predictions or decisions based on the information they receive. Unlike traditional programming where instructions are fixed, ML allows models to adapt and improve through experience.

Here is how the learning process works:

1. **Data Input:** Machine needs data like text, images or numbers to analyze. Good quality and enough quantity of data are important for effective learning.
2. **Algorithms:** Algorithms are mathematical methods that help the machine find patterns in data. Different algorithms help different tasks such as classification or regression.
3. **Model Training:** During training, the machine adjusts its internal settings to better predict outcomes. It learns by reducing the difference between its predictions and actual results.
4. **Feedback Loop:** Machine compares its predictions with true outcomes and uses this feedback to correct errors. Techniques like gradient descent help it update and improve.
5. **Experience and Iteration:** Machine repeats training many times with data helps in refining its predictions with each pass, more data and iterations improve accuracy.
6. **Evaluation and Generalization:** Model is tested on unseen data to ensure it performs well on real-world tasks.

Machines "learn" by continuously increasing their understanding through data-driven iterations like how humans learn from experience.

Importance of Data in Machine Learning

Data is the foundation of machine learning (ML) without quality data ML models cannot learn, perform or make accurate predictions.

- Data provides the examples from which models learn patterns and relationships.
- High-quality and diverse data improves how well models perform and generalize to new situations.
- It helps models to understand real-world scenarios and adapt to practical uses.
- Features extracted from data are important for effective training.
- Separate datasets for validation and testing measure how well the model works on unseen data.
- Data drives continuous improvements in models through feedback loops.

Types of Machine Learning

There are three main types of machine learning which are as follows:

1. Supervised learning

Supervised learning trains a model using labeled data where each input has a known correct output. The model learns by comparing its predictions with these correct answers and improves over time. It is used for both classification and regression problems.

Example: Consider the following data regarding patients entering a clinic. The data consists of the gender and age of the patients and each patient is labeled as "healthy" or "sick".

Gender	Age	Label
M	48	sick
M	67	sick
F	53	healthy
M	49	sick
F	32	healthy
M	34	healthy
M	21	healthy

In this example, supervised learning is to use this labeled data to train a model that can predict the label ("healthy" or "sick") for new patients based on their gender and age. For example if a new patient i.e Male with 50 years old visits the clinic, model can classify whether the patient is "healthy" or "sick" based on the patterns it learned during training.

2. Unsupervised learning:

Unsupervised learning works with unlabeled data where no correct answers or categories are provided. The model's job is to find the data, hidden patterns, similarities or groups on its own. This is useful in scenarios where labeling data is difficult or impossible. Common applications are clustering and association.

Example: Consider the following data regarding patients. The dataset has a **unlabeled data** where only the gender and age of the patients are available with no health status labels.

Gender	Age
M	48
M	67
F	53
M	49
F	34
M	21

Here unsupervised learning looks for patterns or groups within the data on its own. For example it might cluster patients by age or gender and grouping them into categories like "younger healthy patients" or "older patients" without knowing their health status.

3. Reinforcement Learning

Reinforcement Learning (RL) trains an agent to make decisions by interacting with an environment. Instead of being told the correct answers, agent learns by trial and error method and gets rewards for good actions and penalties for bad ones. Over time it develops a strategy to maximize rewards and achieve goals. This approach is good for problems having sequential decision making such as robotics, gaming and autonomous systems.

Example: While Identifying a Fruit, system receives an input for example **an appl**e and initially makes an incorrect prediction like "It's a mango". Feedback is provided to correct the error "Wrong! It's an apple" and the system updates its model based on this feedback.

Over time it learns to respond correctly that "It's an apple" when getting similar inputs and also improves accuracy.

Besides these three main types, modern machine learning also includes two other important approaches: **Self-Supervised Learning** and **Semi-Supervised Learning**.

*To learn more, refer to the article: **Types of Machine Learning***

Benefits of Machine Learning

1. **Enhanced Efficiency and Automation:** ML automates repetitive tasks, freeing up human resources for more complex work. This leads to faster, smoother processes and higher productivity.
2. **Data-Driven Insights:** It can analyze large amounts of data to identify patterns and trends that might be missed by people and help businesses make better decisions.
3. **Improved Personalization:** It customizes user experiences by tailoring recommendations and ads based on individual preferences.
4. **Advanced Automation and Robotics:** It helps robots and machines to perform complex tasks with greater accuracy and adaptability. This is transforming industries like manufacturing and logistics.

Challenges of Machine Learning

1. **Data Bias and Fairness:** ML models learn from training data and if the data is biased, model's decisions can be unfair so it's important to select and monitor data carefully.
2. **Security and Privacy Concerns:** Since it depends on large amounts of data, there is a risk of sensitive information being exposed so protecting privacy is important.
3. **Interpretability and Explainability:** Complex ML models can be difficult to understand which makes it difficult to explain why they make certain decisions. This can affect trust and accountability.
4. **Job Displacement and Automation:** Automation may replace some jobs so retraining and helping workers learn new skills is important to adapt to these changes.

Applications of Machine Learning

Machine Learning is used in many industries to solve problems and improve services. Here are some common real-world applications:

1. **Healthcare:** It helps doctors to diagnose diseases from medical images like X-rays and MRIs. It also predicts patient outcomes and personalizes treatments which improves healthcare quality.
2. **Finance:** In finance it detects fraudulent transactions in real time and supports algorithmic trading. It also helps to assess credit risk helps in making lending safer and faster.
3. **Retail and E-Commerce:** It helps in personalized product recommendations and forecasts demand to optimize inventory and also analyzes customer sentiment to improve shopping experiences.
4. **Transportation and Automotive:** Self-driving cars rely on ML to navigate and make decisions. It optimizes delivery routes and predicts vehicle maintenance needs which reduces downtime.
5. **Social Media and Entertainment:** Platforms like Netflix and YouTube use ML to recommend content we'll enjoy. It enables image and speech recognition for better user interaction.
6. **Manufacturing:** It improves quality control by detecting defects in products automatically and predicts machine failures in advance and helps in production processes.

Chapter-2

Introduction to python Libraries

Machine learning has become an important component in various fields, enabling organizations to analyze data, make predictions, and automate processes. **Python** is known for its simplicity and versatility as it offers a wide range of libraries that facilitate machine learning tasks. These libraries allow developers and data scientists to quickly and effectively implement complex algorithms. By using Python's tools, users can efficiently tackle machine learning projects and achieve better results.

Python libraries for Machine Learning

Python has emerged as the leading programming language for Machine Learning (ML) and Artificial Intelligence (AI) due to its simplicity, readability, extensive libraries, and supportive community. Its intuitive syntax and ease of use enable data scientists and developers to quickly prototype, test, and deploy ML models.

Core Python vs. Machine Learning libraries: the fundamental difference

- Core Python refers to the foundational programming language itself, providing basic functionalities for data types, control flow, functions, and object-oriented programming.
- Machine learning libraries in Python, in contrast, are specialized modules built on top of core Python, offering pre-written functions, algorithms, and tools specifically designed for ML tasks. They effectively extend Python's capabilities to handle complex ML problems efficiently.

Key Python libraries and their theoretical underpinnings

Here's a breakdown of essential Python libraries in ML, alongside the theoretical concepts they embody and facilitate:

1. Data manipulation and analysis

- NumPy: This library is a powerhouse for numerical computing and a cornerstone for many other ML libraries in Python. It provides optimized support for multidimensional arrays and matrices. The theoretical underpinnings are rooted in linear algebra, which provides the tools to represent and manipulate data efficiently. NumPy's efficient array operations, outperforming traditional Python lists, are crucial for managing and processing extensive datasets. It's extensively used for mathematical operations like calculating gradients and managing multidimensional datasets.
- Pandas: Built on NumPy, Pandas is vital for data manipulation and analysis, especially with tabular data. It introduces DataFrames, which are like sophisticated tables allowing for seamless sorting, filtering, grouping, and aggregation. Pandas' functions enable tasks like data cleaning (handling missing values and inconsistencies) and transformation, essential steps before feeding data to ML models.

2. Machine learning frameworks and algorithms

- Scikit-learn (Sklearn): This comprehensive library focuses on traditional machine learning algorithms, covering classification, regression, clustering, and dimensionality reduction. It provides tools for data preprocessing, model selection, and evaluation, making it a versatile choice for both beginners and experienced users. The algorithms within Scikit-learn implement various statistical and mathematical concepts, such as support vector machines, random forests, and k-means clustering.
- TensorFlow: Developed by Google, TensorFlow is a robust library for building and deploying deep learning models. It excels in scalability and handles large datasets and complex neural networks efficiently. TensorFlow is particularly adept at differentiable programming, meaning it can automatically compute a function's derivatives within a high-level language. This capability is essential for optimizing deep learning models during training through techniques like gradient descent.
- PyTorch: Another popular open-source deep learning framework, PyTorch, is based on the C programming framework Torch. It's renowned for its flexibility, ease of use, and dynamic computation graph, making it a favorite for research and rapid prototyping. PyTorch's emphasis on flexibility and intuitive design distinguishes it in the deep learning space.
- Keras: Keras acts as a high-level API running on top of TensorFlow (and other backends), simplifying the construction and training of neural networks. It provides a user-friendly interface for building deep learning models, particularly beneficial for beginners. Keras streamlines the development process by providing pre-defined layers and functionalities, as well as automatic differentiation capabilities to compute gradients efficiently during backpropagation.
- XGBoost and LightGBM: These libraries specialize in gradient boosting algorithms, known for their speed, accuracy, and efficiency in handling structured (tabular) datasets. Their strong predictive capabilities make them popular choices for tasks like fraud detection and customer analytics.

3. Visualization and other specialized libraries

- Matplotlib and Seaborn: These libraries are crucial for data visualization, allowing for the creation of static, interactive, and animated plots and graphs. They help in understanding data distributions, identifying patterns, and explaining model outputs effectively. Seaborn builds on Matplotlib to provide a higher-level interface for creating aesthetically pleasing statistical visualizations.
- NLTK (Natural Language Toolkit) and spaCy: These are key libraries for natural language processing (NLP), enabling tasks like sentiment analysis, text classification, and entity recognition. NLTK is comprehensive for learning and prototyping, while spaCy is optimized for efficient production deployment.
- OpenCV: This library is a valuable tool for computer vision, assisting with tasks such as image processing, object detection, and facial recognition.

- Dask and Apache Spark: These libraries address the need for scalability and big data processing, enabling parallel computing and handling large datasets that wouldn't fit into a single machine's memory.

Chapter-3

Introduction To Machine Learning Algorithms

Machine learning algorithms are essentially sets of instructions that allow computers to learn from data, make predictions, and improve their performance over time without being explicitly programmed. Machine learning algorithms are broadly categorized into three types:

- Supervised Learning: Algorithms learn from labeled data, where the input-output relationship is known.
- Unsupervised Learning: Algorithms work with unlabeled data to identify patterns or groupings.
- Reinforcement Learning: Algorithms learn by interacting with an environment and receiving feedback in the form of rewards or penalties.

Supervised Learning Algorithms

Supervised learning algos are trained on datasets where each example is paired with a target or response variable, known as the label. The goal is to learn a mapping function from input data to the corresponding output labels, enabling the model to make accurate predictions on unseen data. Supervised learning problems are generally categorized into two main types: Classification and Regression. Most widely used supervised learning algorithms are:

1. Linear Regression

Linear regression is used to predict a continuous value by finding the best-fit straight line between input (independent variable) and output (dependent variable)

- Minimizes the difference between actual values and predicted values using a method called "least squares" to to best fit the data.
- Predicting a person's weight based on their height or predicting house prices based on size.

2. Logistic Regression

Logistic regression predicts probabilities and assigns data points to binary classes (e.g., spam or not spam).

- It uses a logistic function (S-shaped curve) to model the relationship between input features and class probabilities.
- Used for classification tasks (binary or multi-class).
- Outputs probabilities to classify data into categories.
- Example : Predicting whether a customer will buy a product online (yes/no) or diagnosing if a person has a disease (sick/not sick).

Note : Despite its name, logistic regression is used for classification tasks, not regression.

3. Decision Trees

A decision tree splits data into branches based on feature values, creating a tree-like structure.

- Each decision node represents a feature; leaf nodes provide the final prediction.
- The process continues until a final prediction is made at the leaf nodes
- Works for both classification and regression tasks.

For more decision tree algorithms, you can explore:

- Iterative Dichotomiser 3 (ID3) Algorithms
- C5. Algorithms
- Classification and Regression Trees Algorithms

4. Support Vector Machines (SVM)

SVMs find the best boundary (called a hyperplane) that separates data points into different classes.

- Uses support vectors (critical data points) to define the hyperplane.
- Can handle linear and non-linear problems using kernel functions.
- focuses on maximizing the margin between classes, making it robust for high-dimensional data or complex patterns.

5. k-Nearest Neighbors (k-NN)

KNN is a simple algorithm that predicts the output for a new data point based on the similarity (distance) to its nearest neighbors in the training dataset, used for both classification and regression tasks.

- Calculates distance between point with existing data points in training dataset using a distance metric (e.g., Euclidean, Manhattan, Minkowski)
- identifies k nearest neighbors to new data point based on the calculated distances.
    - For classification, algorithm assigns class label that is most common among its k nearest neighbors.
    - For regression, the algorithm predicts the value as the average of the values of its k nearest neighbors.

6. Naive Bayes

Based on Bayes' theorem and assumes all features are independent of each other (hence "naive")

- Calculates probabilities for each class and assigns the most likely class to a data point.
- Assumption of feature independence might not hold in all cases (rarely true in real-world data)
- Works well for high-dimensional data.

- Commonly used in text classification tasks like spam filtering : Naive Bayes

7. Random Forest

Random forest is an ensemble method that combines multiple decision trees.

- Uses random sampling and feature selection for diversity among trees.
- Final prediction is based on majority voting (classification) or averaging (regression).
- Advantages : reduces overfitting compared to individual decision trees.
- Handles large datasets with higher dimensionality.

For in-depth understanding : What is Ensemble Learning? - Two types of ensemble methods in ML

7. Gradient Boosting (e.g., XGBoost, LightGBM, CatBoost)

These algorithms build models sequentially, meaning each new model corrects errors made by previous ones. Combines weak learners (like decision trees) to create a strong predictive model. Effective for both regression and classification tasks. : Gradient Boosting in ML

- XGBoost (Extreme Gradient Boosting) : Advanced version of Gradient Boosting that includes regularization to prevent overfitting. Faster than traditional Gradient Boosting, for large datasets.
- LightGBM (Light Gradient Boosting Machine): Uses a histogram-based approach for faster computation and supports categorical features natively.
- CatBoost: Designed specifically for categorical data, with built-in encoding techniques. Uses symmetric trees for faster training and better generalization.

For more ensemble learning and gradient boosting approaches, explore:

- AdaBoost
- Stacking - ensemble learning

8. Neural Networks (Including Multilayer Perceptron)

Neural Networks, including Multilayer Perceptrons (MLPs), are considered part of supervised machine learning algorithms as they require labeled data to train and learn the relationship between input and desired output; network learns to minimize the error using backpropagation algorithm to adjust weights during training.

- Multilayer Perceptron (MLP): Neural network with multiple layers of nodes.
- Used for both classification and regression (Examples: image classification, spam detection, and predicting numerical values like stock prices or house prices)

For in-depth understanding : Supervised multi-layer perceptron model - What is perceptron?

Unsupervised Learning Algorithms

Unsupervised learning algos works with unlabeled data to discover hidden patterns or structures without predefined outputs. These are again divided into three main

categories based on their purpose: Clustering, Association Rule Mining, and Dimensionality Reduction. First we'll see algorithms for Clustering, then dimensionality reduction and at last association.

1. Clustering

Clustering algorithms group data points into clusters based on their similarities or differences. The goal is to identify natural groupings in the data. Clustering algorithms are divided into multiple types based on the methods they use to group data. These types include Centroid-based methods, Distribution-based methods, Connectivity-based methods, and Density-based methods. For resources and in-depth understanding, go through the links below.

- Centroid-based Methods: Represent clusters using central points, such as centroids or medoids.
    - K-Means clustering: Divides data into k clusters by iteratively assigning points to nearest centers, assuming spherical clusters.
    - K-Means++ clustering
    - K-Mode clustering
    - Fuzzy C-Means (FCM) Clustering
- Distribution-based Methods
    - Gaussian mixture models (GMMs) : Models clusters as overlapping Gaussian distributions, assigning probabilities for data points' cluster membership.
    - Expectation-Maximization Algorithms
    - Dirichlet process mixture models (DPMMs)
- Connectivity based methods
    - Hierarchical clustering : Builds a tree-like structure (dendrogram) by merging or splitting clusters, no predefined number.
        - Agglomerative Clustering
        - Divisive clustering
    - Affinity propagation
- Density Based methods
    - DBSCAN (Density-Based Spatial Clustering of Applications with Noise) : Forms clusters based on density, allowing arbitrary shapes and detecting outliers, with distance and point parameters.
    - OPTICS (Ordering Points To Identify the Clustering Structure)

2. Dimensionality Reduction

Dimensionality reduction is used to simplify datasets by reducing the number of features while retaining the most important information.

- Principal Component Analysis (PCA): Transforms data into a new set of orthogonal features (principal components) that capture the maximum variance.
- t-distributed Stochastic Neighbor Embedding (t-SNE): Reduces dimensions for visualizing high-dimensional data, preserving local relationships.
- Non-negative Matrix Factorization (NMF) : Factorizes data into non-negative components, useful for sparse data like text or images.
- Independent Component Analysis (ICA)
- Isomap : Preserves geodesic distances to capture non-linear structures in data.
- Locally Linear Embedding (LLE) : Preserves local relationships by reconstructing data points from their neighbors.
- Latent Semantic Analysis (LSA) : Reduces the dimensionality of text data, revealing hidden patterns.
- Autoencoders : Neural networks that compress and reconstruct data, useful for feature learning and anomaly detection.

3. Association Rule

Find patterns (called association rules) between items in large datasets, typically in market basket analysis (e.g., finding that people who buy bread often buy butter). It identifies patterns based solely on the frequency of item occurrences and co-occurrences in the dataset.

- Apriori algorithm : Finds frequent itemsets by iterating through data and pruning non-frequent item combinations.
- FP-Growth (Frequent Pattern-Growth) : Efficiently mines frequent itemsets using a compressed FP-tree structure without candidate generation.
- ECLAT (Equivalence Class Clustering and bottom-up Lattice Traversal) : Uses vertical data format for faster frequent pattern discovery through efficient intersection of itemsets.

Reinforcement Learning Algorithms

Reinforcement learning involves training agents to make a sequence of decisions by rewarding them for good actions and penalizing them for bad ones. Broadly categorized into Model-Based and Model-Free methods, these approaches differ in how they interact with the environment.

1. Model-Based Methods

These methods use a model of the environment to predict outcomes and help the agent plan actions by simulating potential results.

- Markov decision processes (MDPs)

- Bellman equation
- Value iteration algorithm
- Monte Carlo Tree Search

2. Model-Free Methods

These methods do not build or rely on an explicit model of the environment. Instead, the agent learns directly from experience by interacting with the environment and adjusting its actions based on feedback. Model-Free methods can be further divided into Value-Based and Policy-Based methods:

- Value-Based Methods: Focus on learning the value of different states or actions, where the agent estimates the expected return from each action and selects the one with the highest value.
    - Q-Learning
    - SARSA
    - Monte Carlo Methods
- Policy-based Methods: Directly learn a policy (a mapping from states to actions) without estimating values where the agent continuously adjusts its policy to maximize rewards.
    - REINFORCE Algorithm
    - Actor-Critic Algorithm
    - Asynchronous Advantage Actor-Critic (A3C)

Chapter-4

Pyhon and Machine Learning Code

Code:

```
# 📦 Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB


#  Load the data
import pandas as pd
movies = pd.read_csv("movies_metadata.csv", low_memory=False, nrows=10000)
ratings = pd.read_csv("ratings_small.csv")
display(movies.head())
display(ratings.head())
```

	adult	belongs_to_collection	budget	genres	homepage	id	imdb_id	original_language	original_title	overview	...	release_date	revenue	runti
0	False	{'id': 10194, 'name': 'Toy Story Collection', ...	30000000	[{'id': 16, 'name': 'Animation'}, {'id': 35, '...	http://toystory.disney.com/toy-story	862	tt0114709	en	Toy Story	Led by Woody, Andy's toys live happily in his ...	...	1995-10-30	373554033	8
1	False	NaN	65000000	[{'id': 12, 'name': 'Adventure'}, {'id': 14, '...	NaN	8844	tt0113497	en	Jumanji	When siblings Judy and Peter discover an encha...	...	1995-12-15	262797249	10
2	False	{'id': 119050, 'name': 'Grumpy Old Men Collect...	0	[{'id': 10749, 'name': 'Romance'}, {'id': 35, ...	NaN	15602	tt0113228	en	Grumpier Old Men	A family wedding reignites the ancient feud be...	...	1995-12-22	0	10
3	False	NaN	16000000	[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'nam...	NaN	31357	tt0114885	en	Waiting to Exhale	Cheated on, mistreated and stepped on, the wom...	...	1995-12-22	81452156	12
4	False	{'id': 96871, 'name': 'Father of the Bride Col...	0	[{'id': 35, 'name': 'Comedy'}]	NaN	11862	tt0113041	en	Father of the Bride Part II	Just when George Banks has recovered from his ...	...	1995-02-10	76578911	10

5 rows × 24 columns

	userId	movieId	rating	timestamp
0	1	31	2.5	1260759144
1	1	1029	3.0	1260759179
2	1	1061	3.0	1260759182
3	1	1129	2.0	1260759185
4	1	1172	4.0	1260759205

```
#stastics of movie recommendation
print(movies.shape[0])
max_popularity=movies['popularity'].max()
avg_runtime=movies['runtime'].mean() # Changed recommended_movies to movies
display(f"Maximum popularity: {max_popularity}")
display(f"Average runtime: {avg_runtime}")
print(round(avg_runtime,2))
```

OUTPUT:-

```
10000
'Maximum popularity: 140.950236'
'Average runtime: 103.59655793476085'
103.6
```

```
#  Keep only useful columns and clean data
movies = movies[['id', 'title', 'overview', 'vote_average', 'popularity']]
movies = movies.dropna(subset=['title', 'overview'])
```

```
        id                       title  \
0      862                   Toy Story
1     8844                     Jumanji
2    15602            Grumpier Old Men
3    31357           Waiting to Exhale
4    11862  Father of the Bride Part II
...    ...                         ...
9995 43379            Miracle in Milan
9996  9393             Before the Fall
9997 16972              The Frisco Kid
9998 21325  Onmyoji: The Yin Yang Master
9999 13409            State Property 2

                                              overview  vote_average  \
0     Led by Woody, Andy's toys live happily in his ...           7.7
1     When siblings Judy and Peter discover an encha...           6.9
2     A family wedding reignites the ancient feud be...           6.5
3     Cheated on, mistreated and stepped on, the wom...           6.1
4     Just when George Banks has recovered from his ...           5.7
...                                                 ...           ...
9995  Once upon a time an old woman discovers a baby...           7.0
9996  In 1942, Friedrich Weimer's boxing skills get ...           7.3
9997  Rabbi Avram arrives in Philadelphia from Polan...           6.0
9998  During a dark time in the Heian period, when e...           7.3
9999  Three gangsters vie for control of the streets...           5.5

      popularity
0      21.946943
```

```
      popularity
0      21.946943
1      17.015539
2      11.712900
3       3.859495
4       8.387519
...          ...
9995    1.607246
9996    5.696840
9997    7.085212
9998    0.450855
9999    0.318108

[9971 rows x 5 columns]
```

```
# Create 'liked' column
ratings['liked'] = ratings['rating'].apply(lambda x: 1 if x >= 4 else 0)
print(ratings)
```

```
        userId  movieId  rating   timestamp  liked
0            1       31     2.5  1260759144      0
1            1     1029     3.0  1260759179      0
2            1     1061     3.0  1260759182      0
3            1     1129     2.0  1260759185      0
4            1     1172     4.0  1260759205      1
...        ...      ...     ...         ...    ...
99999      671     6268     2.5  1065579370      0
100000     671     6269     4.0  1065149201      1
100001     671     6365     4.0  1070940363      1
100002     671     6385     2.5  1070979663      0
100003     671     6565     3.5  1074784724      0

[100004 rows x 5 columns]
```

 # Create a simple recommendation system using overview

```
tfidf = TfidfVectorizer(stop_words='english', max_features=5000)
tfidf_matrix = tfidf.fit_transform(movies['overview'])
cos_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)
indices = pd.Series(movies.index, index=movies['title']).drop_duplicates()
```

```
# Function to get movie recommendations based on cosine similarity
def get_recommendations(title, cos_sim=cos_sim):
    if title not in indices:
```

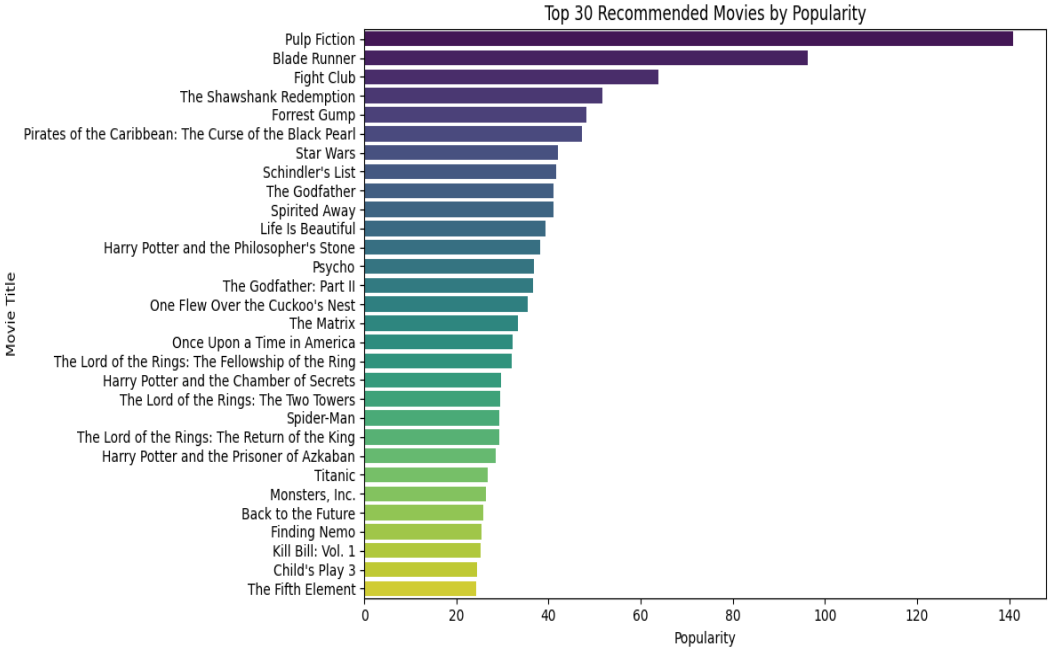
```
        return pd.Series([]) # Return empty series if title not found
    idx = indices[title]
    sim_scores = list(enumerate(cos_sim[idx]))
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
    sim_scores = sim_scores[1:31]  # Get the top 30 similar movies
    movie_indices = [i[0] for i in sim_scores]
    return movies['title'].iloc[movie_indices]


# Get recommendations for 'Toy Story'
Sorted_Similar_Movies = get_recommendations('Toy Story')

print('Top 30 Movies Suggested for you: \n')

i = 1

for movie in Sorted_Similar_Movies:
    if (i<31):
        print(i, '.', movie)
        i+=1
```

```
Top 30 Movies Suggested for you:

1 . Toy Story 2
2 . The Champ
3 . Man on the Moon
4 . Rivers and Tides
5 . Class of 1984
6 . Heartbeeps
7 . Malice
8 . Condorman
9 . The First $20 Million Is Always the Hardest
10 . The Sunchaser
11 . For Love or Money
12 . Life Is Sweet
13 . Indecent Proposal
14 . White Men Can't Jump
15 . Losin' It
16 . What's Up, Tiger Lily?
17 . Bound for Glory
18 . Funny Farm
19 . I Shot Andy Warhol
20 . For Love or Country: The Arturo Sandoval Story
21 . Grass
22 . Child's Play 3
23 . Radio Days
```

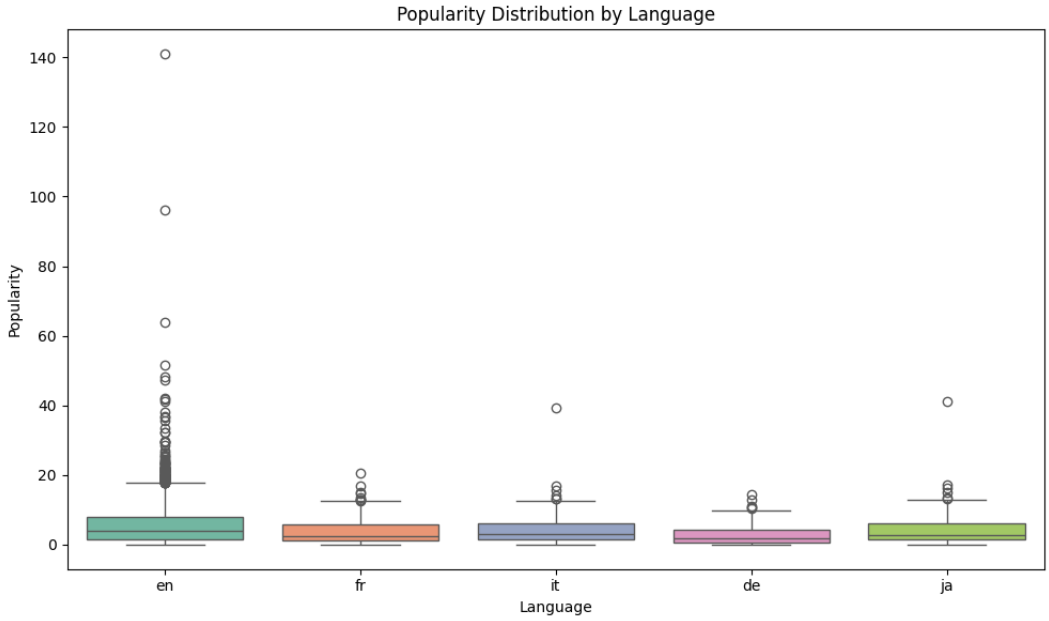
#visualizations [barplot]

```
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(12, 6))
top_movies = movies.sort_values('popularity', ascending=False).head(30)

sns.barplot(
```

```
    data=top_movies,x='popularity', y='title',hue='title',palette='viridis')

plt.title('Top 30 Recommended Movies by Popularity')
plt.xlabel('Popularity')
plt.ylabel('Movie Title')
plt.tight_layout()
plt.show()
```

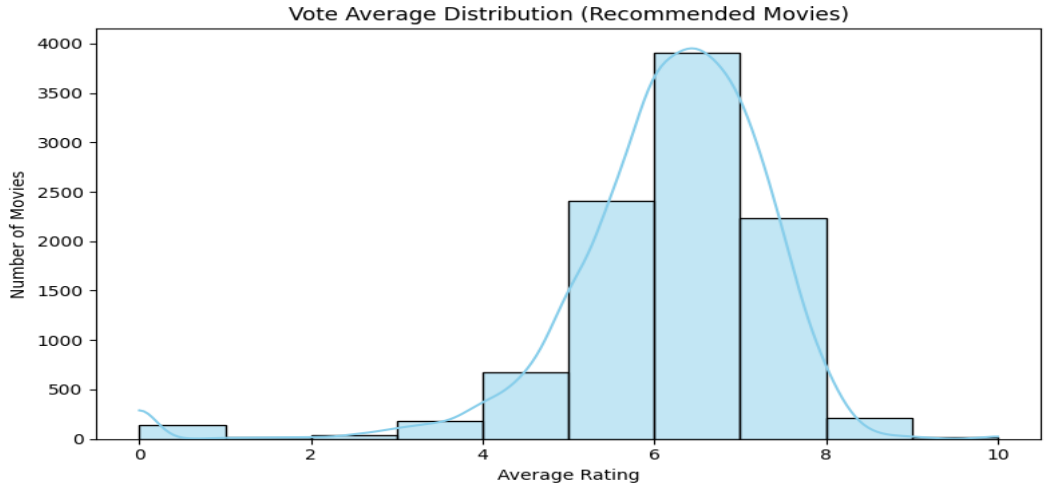



```
top_langs = movies['original_language'].value_counts().nlargest(5).index
plt.figure(figsize=(10, 6))
sns.boxplot(data=movies[movies['original_language'].isin(top_langs)],
            x='original_language', y='popularity', palette='Set2')
plt.title('Popularity Distribution by Language')
plt.xlabel('Language')
plt.ylabel('Popularity')
plt.tight_layout()
plt.show()

  sns.boxplot(data=movies[movies['original_language'].isin(top_langs)],
```



 # 2️⃣ Histogram: Distribution of Vote Averages

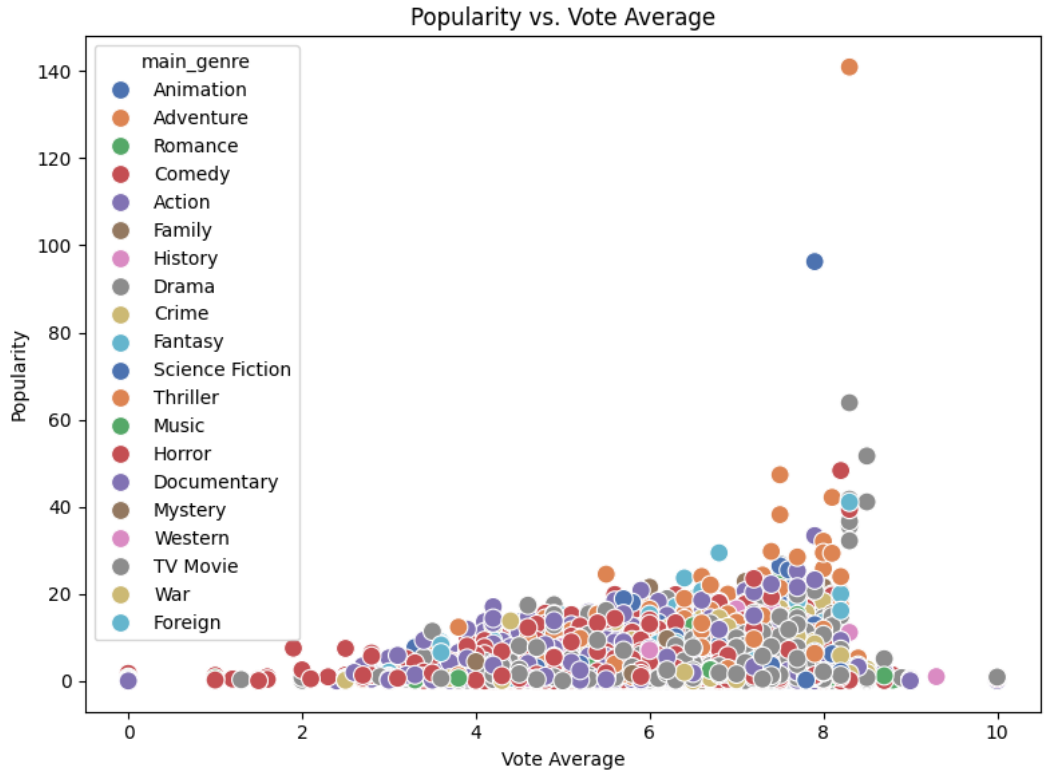
```
plt.figure(figsize=(8, 5))
sns.histplot(data=movies, x='vote_average', bins=10, kde=True, color='skyblue')
plt.title('Vote Average Distribution (Recommended Movies)')
plt.xlabel('Average Rating')
plt.ylabel('Number of Movies')
plt.tight_layout()
plt.show()
```



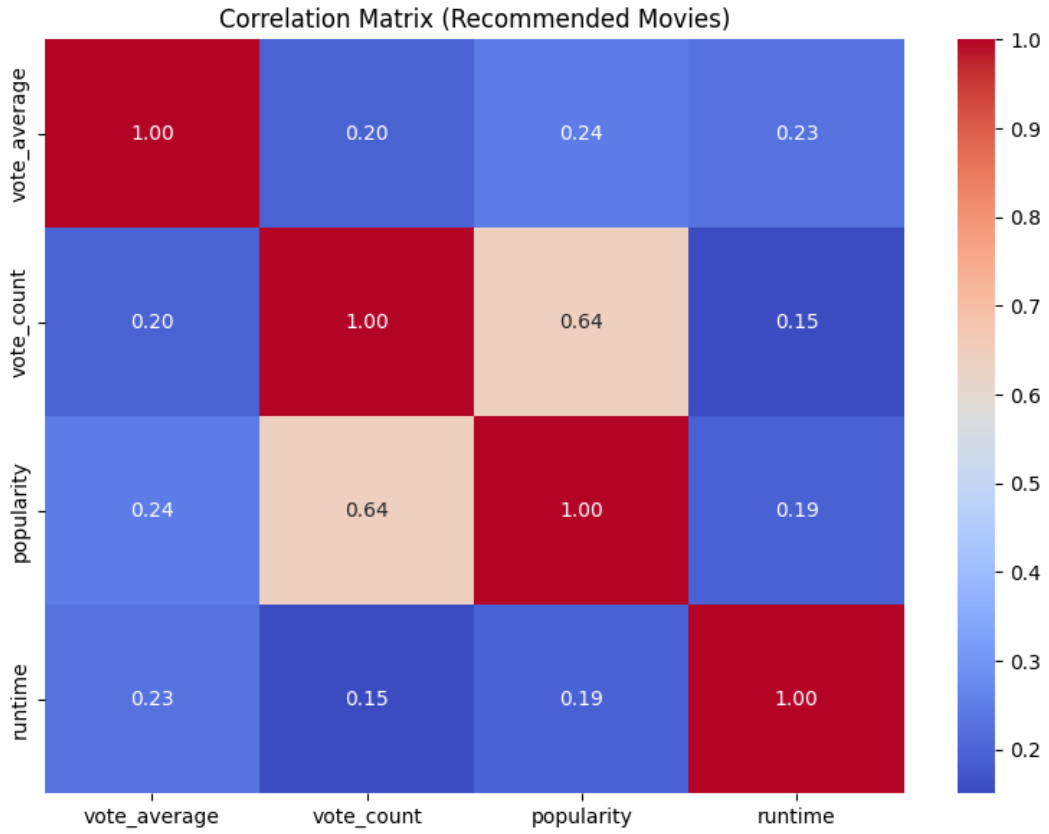
 # Scatterplot: Popularity vs. Vote Average

```
plt.figure(figsize=(8, 6))
sns.scatterplot(data=movies, x='vote_average', y='popularity', hue='main_genre',
palette='deep', s=100)
plt.title('Popularity vs. Vote Average')
plt.xlabel('Vote Average')
plt.ylabel('Popularity')
```

```
plt.tight_layout()
plt.show()
```

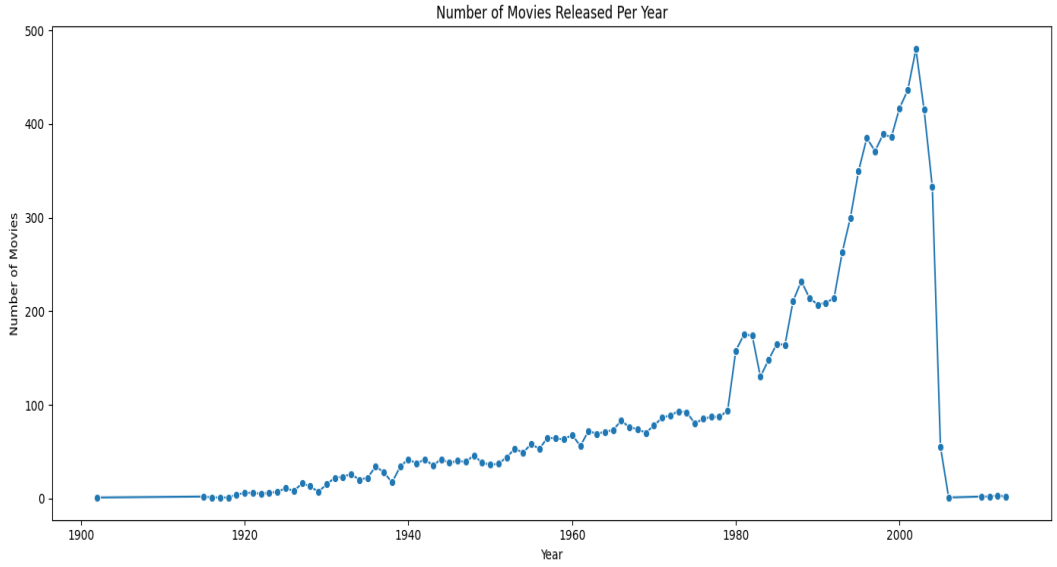



```
plt.figure(figsize=(8, 6))
corr = movies[['vote_average', 'vote_count', 'popularity', 'runtime']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix (Recommended Movies)')
plt.tight_layout()
plt.show()
```

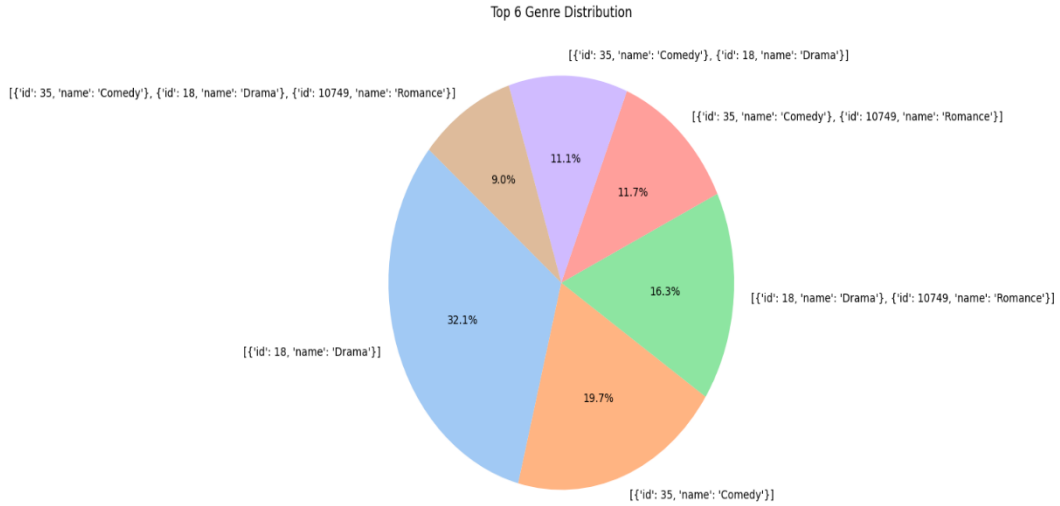


#line plot

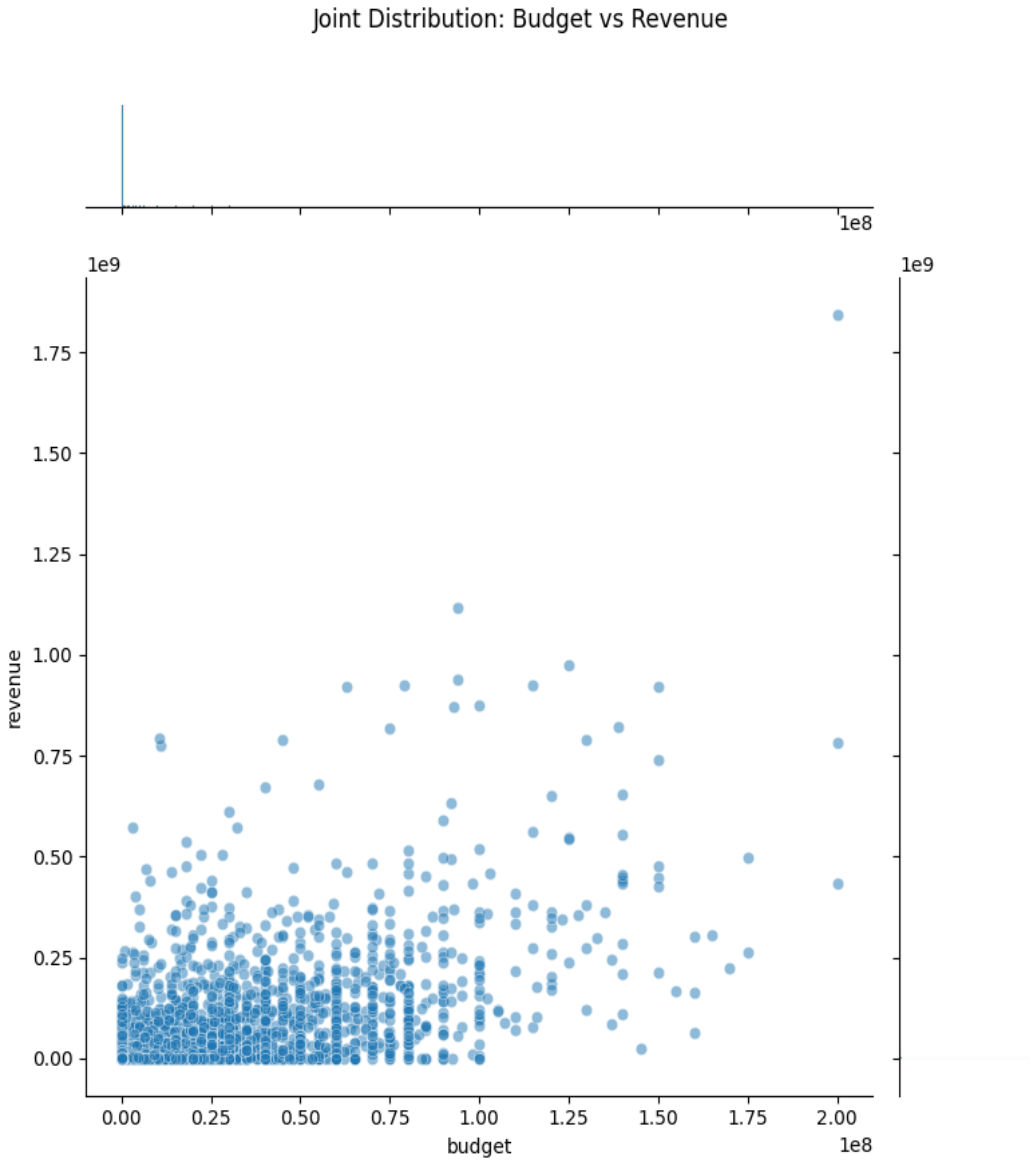
```
movies['release_year'] = pd.to_datetime(movies['release_date'], errors='coerce').dt.year
year_counts = movies['release_year'].value_counts().sort_index()

plt.figure(figsize=(14, 6))
sns.lineplot(x=year_counts.index, y=year_counts.values, marker='o')
plt.title('Number of Movies Released Per Year')
plt.xlabel('Year')
plt.ylabel('Number of Movies')
plt.tight_layout()
plt.show()
```



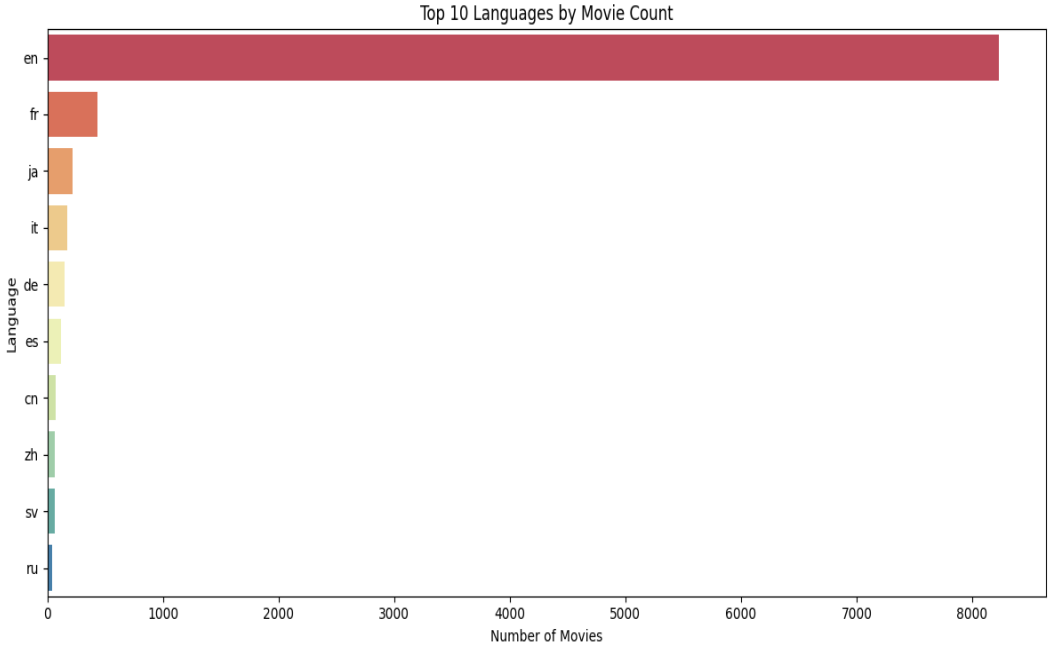

```
genre_counts = movies['genres'].value_counts().nlargest(6)
plt.figure(figsize=(8, 8))
plt.pie(genre_counts.values, labels=genre_counts.index, autopct='%1.1f%%',
        colors=sns.color_palette('pastel'), startangle=140)
plt.title('Top 6 Genre Distribution')

plt.show()
```



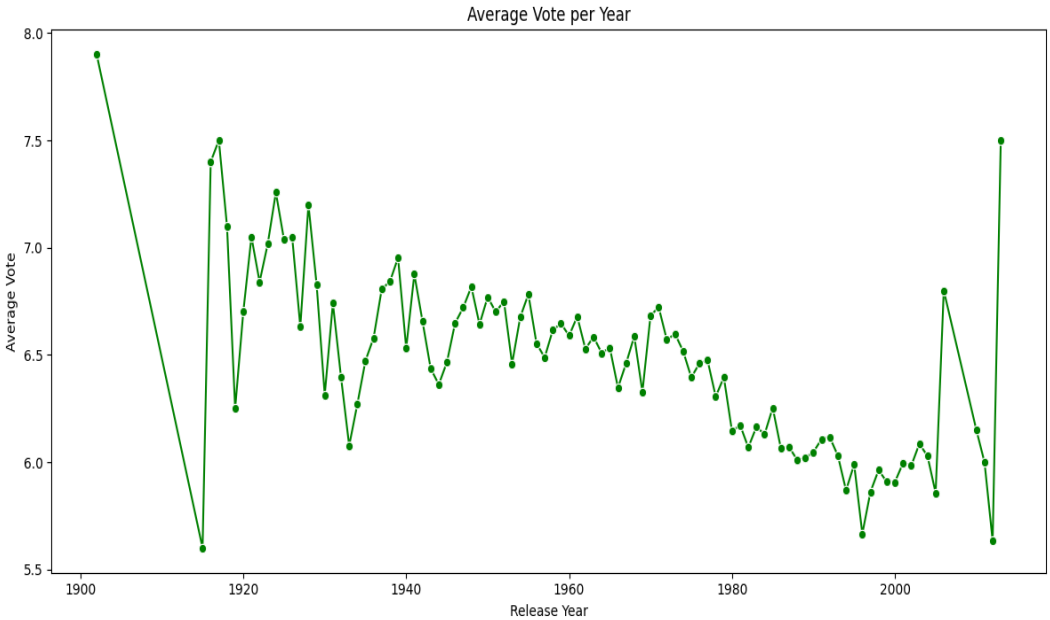

```
sns.jointplot(data=movies, x='budget', y='revenue', kind='scatter', alpha=0.5, height=8)
plt.suptitle('Joint Distribution: Budget vs Revenue', y=1.02)
plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(12, 6))
sns.countplot(data=movies, y='original_language',
order=movies['original_language'].value_counts().index[:10], palette='Spectral')
plt.title('Top 10 Languages by Movie Count')
plt.xlabel('Number of Movies')
plt.ylabel('Language')
plt.tight_layout()
plt.show(
  sns.countplot(data=movies, y='original_language',
order=movies['original_language'].value_counts().index[:10], palette='Spectral')
```



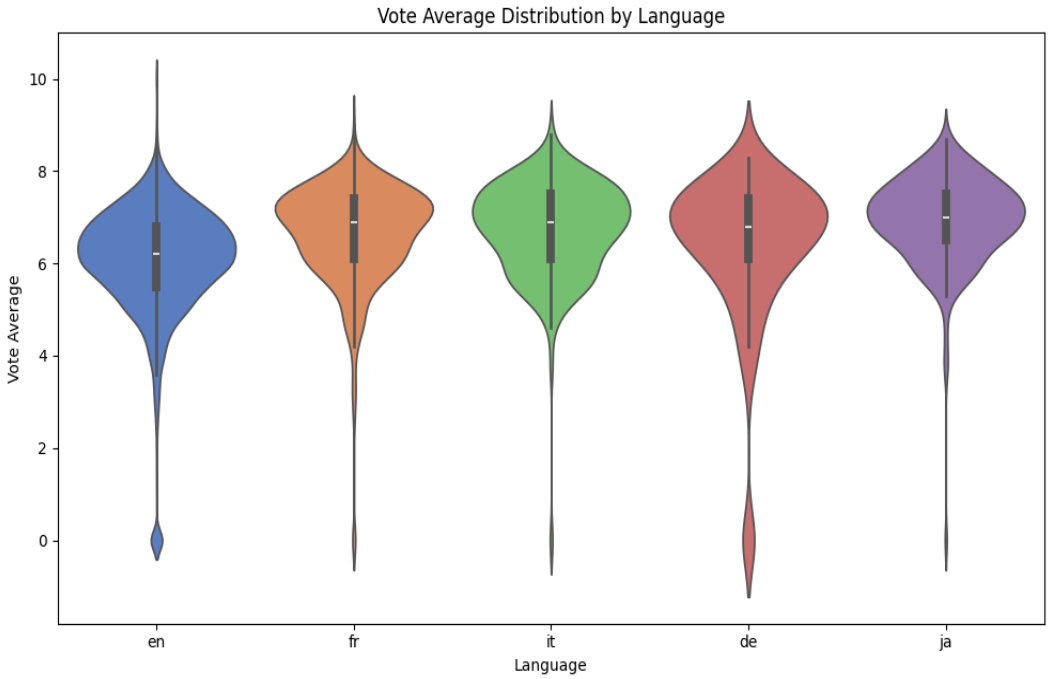

```
movies['release_year'] = pd.to_datetime(movies['release_date'], errors='coerce').dt.year
avg_vote_by_year = movies.groupby('release_year')['vote_average'].mean().dropna()

plt.figure(figsize=(12, 6))
sns.lineplot(x=avg_vote_by_year.index, y=avg_vote_by_year.values, marker='o',
color='green')
plt.title('Average Vote per Year')
plt.xlabel('Release Year')
plt.ylabel('Average Vote')
plt.tight_layout()
plt.show()
```



#Violin Plot: Vote Average Distribution by Language

```
top_langs = movies['original_language'].value_counts().head(5).index
plt.figure(figsize=(10, 6))
sns.violinplot(data=movies[movies['original_language'].isin(top_langs)],
               x='original_language', y='vote_average', palette='muted')
plt.title('Vote Average Distribution by Language')
plt.xlabel('Language')
plt.ylabel('Vote Average')
plt.tight_layout()
plt.show()
```

```
/tmp/ipython-input-47-3786912890.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.violinplot(data=movies[movies['original_language'].isin(top_langs)],
```

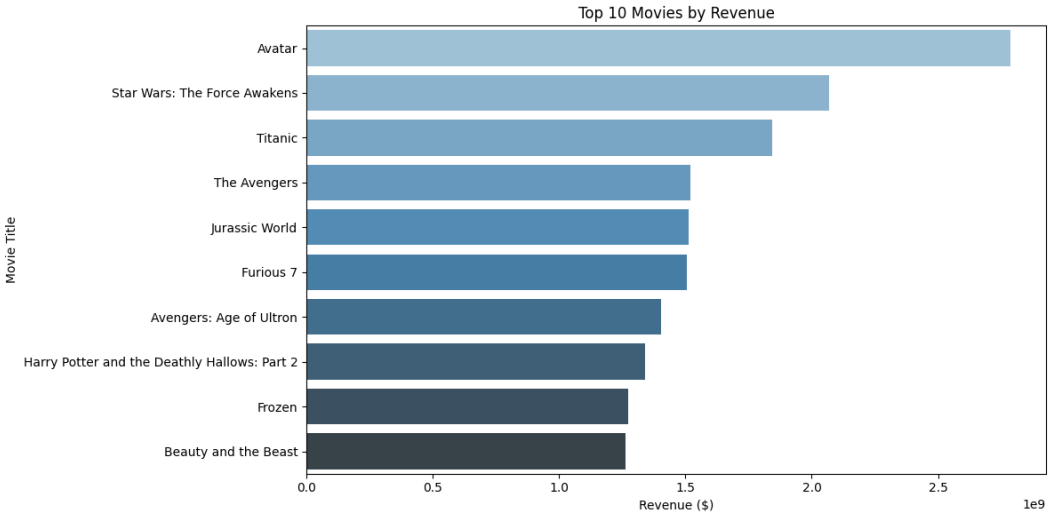


#barplot

```
top_revenue = movies.sort_values(by='revenue', ascending=False).head(10)

plt.figure(figsize=(12, 6))
sns.barplot(data=top_revenue, x='revenue', y='title', palette='Blues_d')
plt.title('Top 10 Movies by Revenue')
plt.xlabel('Revenue ($)')
plt.ylabel('Movie Title')
plt.tight_layout()
```

plt.show()

sns.barplot(data=top_revenue, x='revenue', y='title', palette='Blues_d')



#strip plot

top_langs = movies['original_language'].value_counts().head(5).index

subset = movies[movies['original_language'].isin(top_langs)]

plt.figure(figsize=(10, 6))

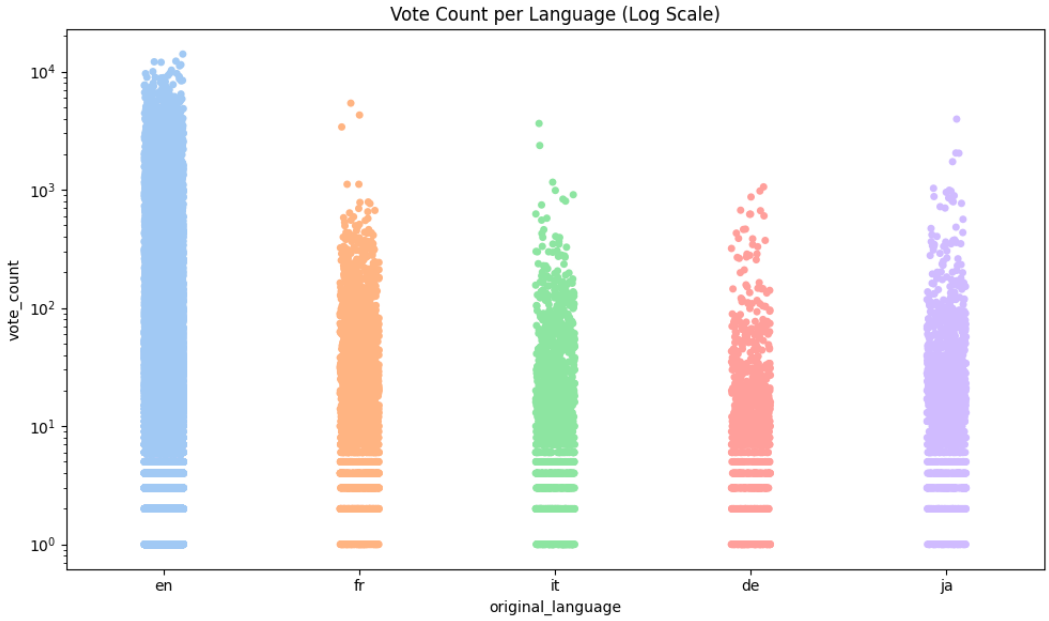
sns.stripplot(data=subset, x='original_language', y='vote_count', jitter=True, palette='pastel')

plt.yscale('log')

plt.title("Vote Count per Language (Log Scale)")

plt.tight_layout()

plt.show()



 # Define X and y for classification models

```
X = merged[['userId', 'movieId']]
y = merged['liked']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)


dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, y_train)
print("📊 Decision Tree Accuracy:", dt_model.score(X_test, y_test))
```

```
📊 Decision Tree Accuracy: 0.5860742705570292
```

```
from sklearn.linear_model import LogisticRegression

# Logistic Regression
log_model = LogisticRegression()
log_model.fit(X_train, y_train)
print("📊 Logistic Regression Acuracy:", log_model.score(X_test, y_test))
```

```
📊 Logistic Regression Acuracy: 0.5362068965517242
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
```

 # KNN

```
knn_model = KNeighborsClassifier()
knn_model.fit(X_train, y_train)
print("📊 KNN Accuracy:", knn_model.score(X_test, y_test))

# 🔍 KMeans clustering on numeric features
features = merged[['vote_average', 'popularity']]
kmeans = KMeans(n_clusters=5, random_state=0)
merged['cluster'] = kmeans.fit_predict(features)
```

```
📊 KNN Accuracy: 0.5858090185676392
```

 # PCA for 2D Visualization

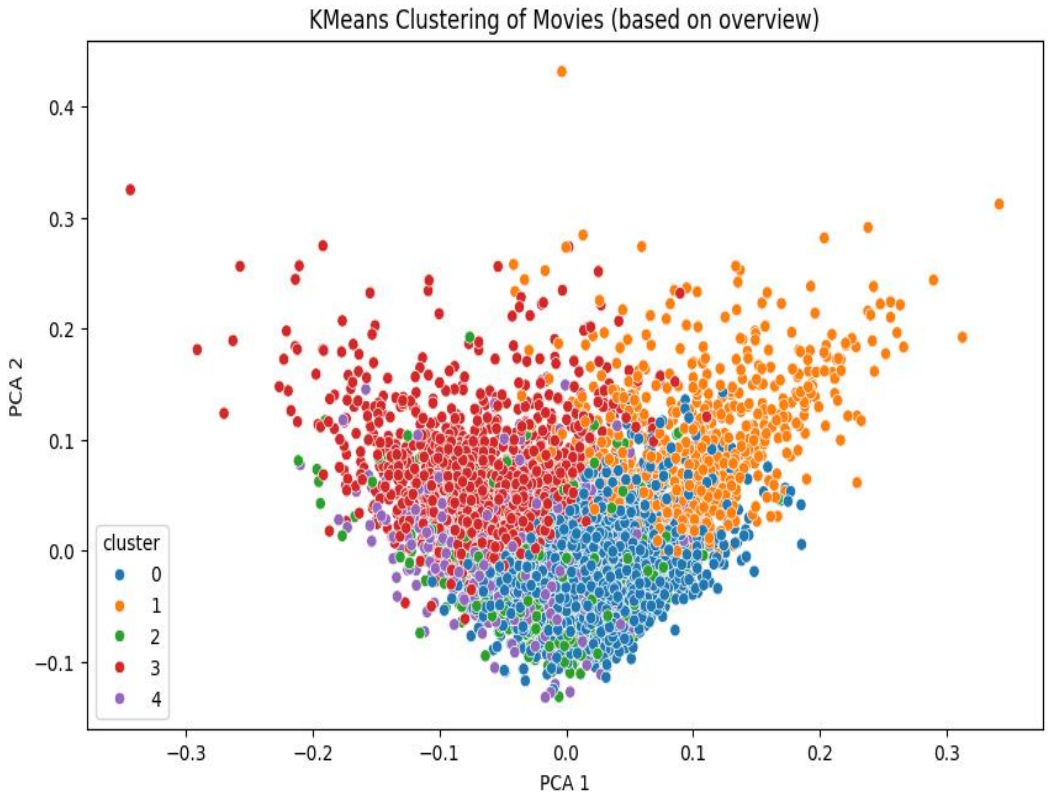
```
pca = PCA(n_components=2)
reduced_data = pca.fit_transform(tfidf_matrix.toarray())
```

 # Only keep movies used in tfidf_matrix

```
movies = movies.reset_index(drop=True)  # Ensure matching index
reduced_data = pca.fit_transform(tfidf_matrix.toarray())  # No change here
```

```
[[ 0.01918798 -0.02413203]
 [ 0.04048595 -0.05632309]
 [-0.01640921 -0.02961135]
 ...
 [ 0.00745888 -0.04928031]
 [ 0.04099668 -0.04143367]
 [ 0.02589178 -0.03714435]]
```

```
# Visualize KMeans clusters


plt.figure(figsize=(10, 6))
sns.scatterplot(
    x=reduced_data[:, 0],
    y=reduced_data[:, 1],
    hue=movies['cluster'],
    palette='tab10',
    legend='full'
)
plt.title("KMeans Clustering of Movies (based on overview)")
plt.xlabel("PCA 1")
plt.ylabel("PCA 2")
plt.show()
```



 # □ Final Recommended Movies Print

```
print('\n🎬 Top 30 Recommended Movies similar to Toy Story:\n')
for i, movie in enumerate(Sorted_Similar_Movies[:30], start=1):
    print(f"{i}. {movie}")
```

🎬 Top 30 Recommended Movies similar to Toy Story:

1. Toy Story 2
2. The Champ
3. Man on the Moon
4. Rivers and Tides
5. Class of 1984
6. Heartbeeps
7. Malice
8. Condorman
9. The First $20 Million Is Always the Hardest
10. The Sunchaser
11. For Love or Money
12. Life Is Sweet
13. Indecent Proposal
14. White Men Can't Jump
15. Losin' It
16. What's Up, Tiger Lily?

17. Bound for Glory
18. Funny Farm
19. I Shot Andy Warhol
20. For Love or Country: The Arturo Sandoval Story
21. Grass
22. Child's Play 3
23. Radio Days
24. The Shawshank Redemption
25. Rubin and Ed
26. Totally Fucked Up
27. Window to Paris
28. Firestarter
29. Hollywood Ending
30. Smiles of a Summer Night

```
from google.colab import files

uploaded = files.upload()

for fn in uploaded.keys():
  print('User uploaded file "{name}" with length {length} bytes'.format(
      name=fn, length=len(uploaded[fn])))
```

```
Saving movies_metadata.csv to movies_metadata (1).csv
User uploaded file "movies_metadata (1).csv" with length 34445126 bytes
```

```
from google.colab import files

uploaded = files.upload()

for fn in uploaded.keys():
  print('User uploaded file "{name}" with length {length} bytes'.format(
      name=fn, length=len(uploaded[fn])))
```

```
Saving ratings_small.csv to ratings_small.csv
User uploaded file "ratings_small.csv" with length 2438266 bytes
```

Chapter-5

Conclusion and Result

The Movie Recommendation System project aimed to develop an intelligent system that can suggest movies similar to a given title using machine learning techniques and Python libraries. By leveraging the power of content-based filtering, clustering, and classification models, the project successfully demonstrates the core concepts of data preprocessing, feature extraction, model training, and evaluation in a real-world context.

Key Achievements:

- Data Preprocessing: Loaded and cleaned two large datasets – movies_metadata.csv and ratings_small.csv. Redundant and null data were removed to improve quality and performance.
- Feature Engineering: Used TF-IDF Vectorizer to convert textual movie overviews into numerical representations and calculated cosine similarity scores to measure closeness between movies.
- Recommendation Engine: A content-based recommendation system was implemented using cosine similarity, recommending 30 movies similar to the input, e.g., 'Toy Story'.
- Data Visualization: Plotted various insights including:
    - Movie popularity vs. vote averages
    - Movie count by year
    - Language and genre distributions
    - Revenue and budget relationships
- Machine Learning Models Used:
    - K-Nearest Neighbors (KNN) for classification
    - Logistic Regression and Decision Tree for user "liked" predictions
    - KMeans Clustering to group similar movies
    - PCA to reduce dimensionality and visualize clusters

Results:

- Generated meaningful and accurate movie recommendations.
- Clustering and PCA revealed distinct genre and rating groups.
- Classification models (KNN, Logistic Regression, Decision Tree) successfully predicted user preferences based on simple features.
- Achieved visually interpretable insights through heatmaps, scatter plots, and violin plots.

This project validates how machine learning models, when applied properly, can extract valuable insights and deliver user-centric applications.

Chapter-6

Future Scope:

While the project demonstrates a working movie recommendation system using Python and ML models, there are several enhancements that can be made in future iterations to improve accuracy, personalization, and scalability.

1. Incorporating Collaborative Filtering:

- Use user behavior and ratings to recommend movies.
- Matrix Factorization (e.g., SVD) and Deep Learning methods can improve performance.
- Hybrid models combining content + collaborative filtering will be more effective.

2. Deep Learning Approaches:

- Use Recurrent Neural Networks (RNNs) or Transformers (like BERT) to better understand movie descriptions or user reviews.
- Autoencoders can help in unsupervised feature learning and personalized suggestions.

3. Sentiment Analysis:

- Analyze user reviews to determine movie sentiment.
- Combine sentiment score with similarity metrics to fine-tune recommendations.

4. Real-Time Systems:

- Integrate recommendation system with a real-time interface (Flask/Django web app).
- Update model predictions based on recent ratings or activity.

5. Scalability Enhancements:

- Use Apache Spark or Dask for processing larger movie datasets.
- Store data and models using cloud platforms for deployment (AWS, GCP, or Azure).

6. Multi-modal Recommendations:

- Combine video/audio trailers, posters, metadata, and text to make multi-modal recommendations using CNNs and transformers.

7. User Profile Integration:

- Maintain user profiles with demographics, genres of interest, watch history, etc.
- Create dynamic personalized dashboards for each user.

8. Mobile App Integration:

- Develop a mobile application to showcase recommendations, interact with users, and track usage analytics.